

OSSP

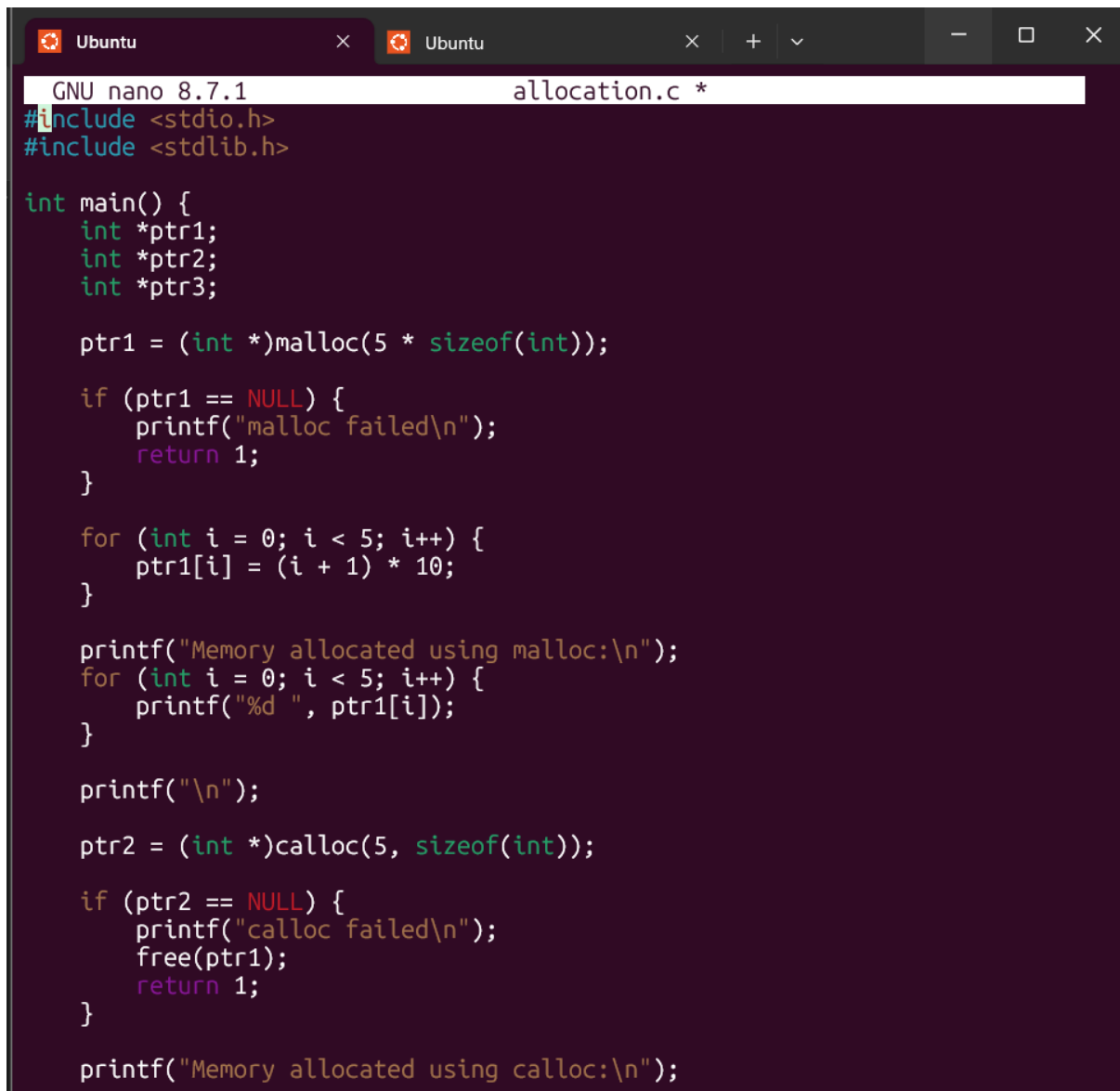
Practical Week -8

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1-:

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.



```
GNU nano 8.7.1 allocation.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr1;
    int *ptr2;
    int *ptr3;

    ptr1 = (int *)malloc(5 * sizeof(int));

    if (ptr1 == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        ptr1[i] = (i + 1) * 10;
    }

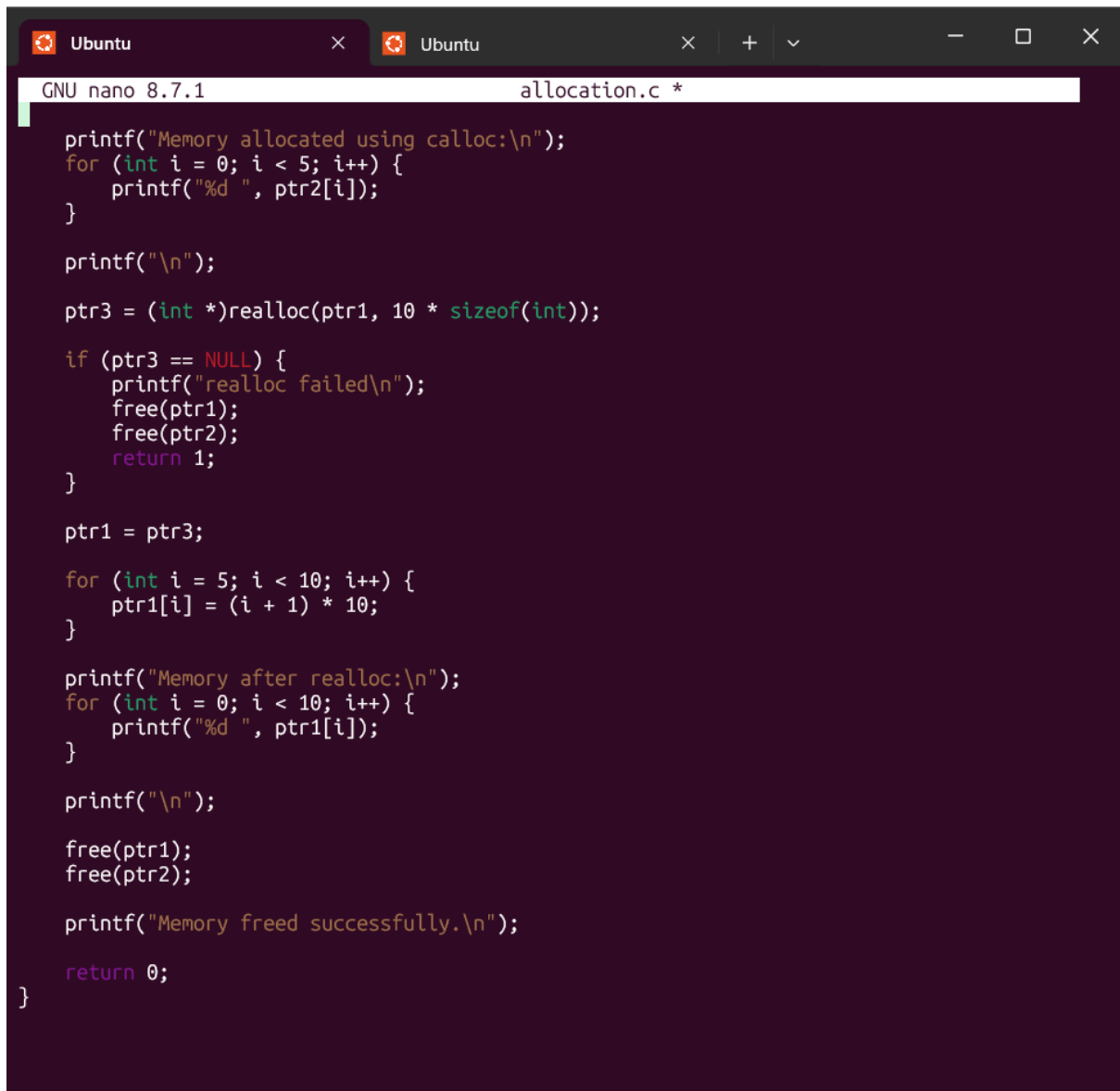
    printf("Memory allocated using malloc:\n");
    for (int i = 0; i < 5; i++) {
        printf("%d ", ptr1[i]);
    }

    printf("\n");

    ptr2 = (int *)calloc(5, sizeof(int));

    if (ptr2 == NULL) {
        printf("calloc failed\n");
        free(ptr1);
        return 1;
    }

    printf("Memory allocated using calloc:\n");
```



The image shows a terminal window with two tabs, both labeled 'Ubuntu'. The active tab is displaying the GNU nano 8.7.1 text editor editing a file named 'allocation.c'. The code in the file is a C program that demonstrates memory allocation and deallocation. It starts by printing 'Memory allocated using calloc:\n'. Then it loops from i=0 to i=4, printing the value of ptr2[i]. After a newline, it reallocates ptr1 to 10 * sizeof(int) and stores the result in ptr3. It then checks if ptr3 is NULL; if so, it prints 'realloc failed\n', frees ptr1 and ptr2, and returns 1. Otherwise, it sets ptr1 = ptr3 and loops from i=5 to i=9, printing the value of ptr1[i]. After another newline, it frees ptr1 and ptr2, prints 'Memory freed successfully.\n', and returns 0.

```
printf("Memory allocated using calloc:\n");
for (int i = 0; i < 5; i++) {
    printf("%d ", ptr2[i]);
}

printf("\n");

ptr3 = (int *)realloc(ptr1, 10 * sizeof(int));

if (ptr3 == NULL) {
    printf("realloc failed\n");
    free(ptr1);
    free(ptr2);
    return 1;
}

ptr1 = ptr3;

for (int i = 5; i < 10; i++) {
    ptr1[i] = (i + 1) * 10;
}

printf("Memory after realloc:\n");
for (int i = 0; i < 10; i++) {
    printf("%d ", ptr1[i]);
}

printf("\n");

free(ptr1);
free(ptr2);

printf("Memory freed successfully.\n");

return 0;
}
```

```
nocturnal@nocturnal:~$ nano allocation.c
nocturnal@nocturnal:~$ gcc allocation.c -o allocation
nocturnal@nocturnal:~$ ./allocation
Memory allocated using malloc:
10 20 30 40 50
Memory allocated using calloc:
0 0 0 0 0
Memory after realloc:
10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
nocturnal@nocturnal:~$
```

```
nocturnal@nocturnal:~$ valgrind --leak-check=full ./allocation
==2032== Memcheck, a memory error detector
==2032== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2032== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2032== Command: ./allocation
==2032==
Memory allocated using malloc:
10 20 30 40 50
Memory allocated using calloc:
0 0 0 0 0
Memory after realloc:
10 20 30 40 50 60 70 80 90 100
Memory freed successfully.
==2032==
==2032== HEAP SUMMARY:
==2032==    in use at exit: 0 bytes in 0 blocks
==2032==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==2032==
==2032== All heap blocks were freed -- no leaks are possible
==2032==
==2032== For lists of detected and suppressed errors, rerun with: -s
==2032== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
nocturnal@nocturnal:~$
```

Task-2:

Develop a program that demonstrates Copy-on-Write behavior after a fork(). Analyze memory usage before and after modifying data in the child process.

```
GNU nano 8.7.1 COW.C *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main() {
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL) {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++) {
        data[i] = i;
    }

    printf("Parent PID: %d\n", getpid());
    printf("Memory allocated: %lu bytes\n",
           (unsigned long)(SIZE * sizeof(int)));

    pid_t pid = fork();

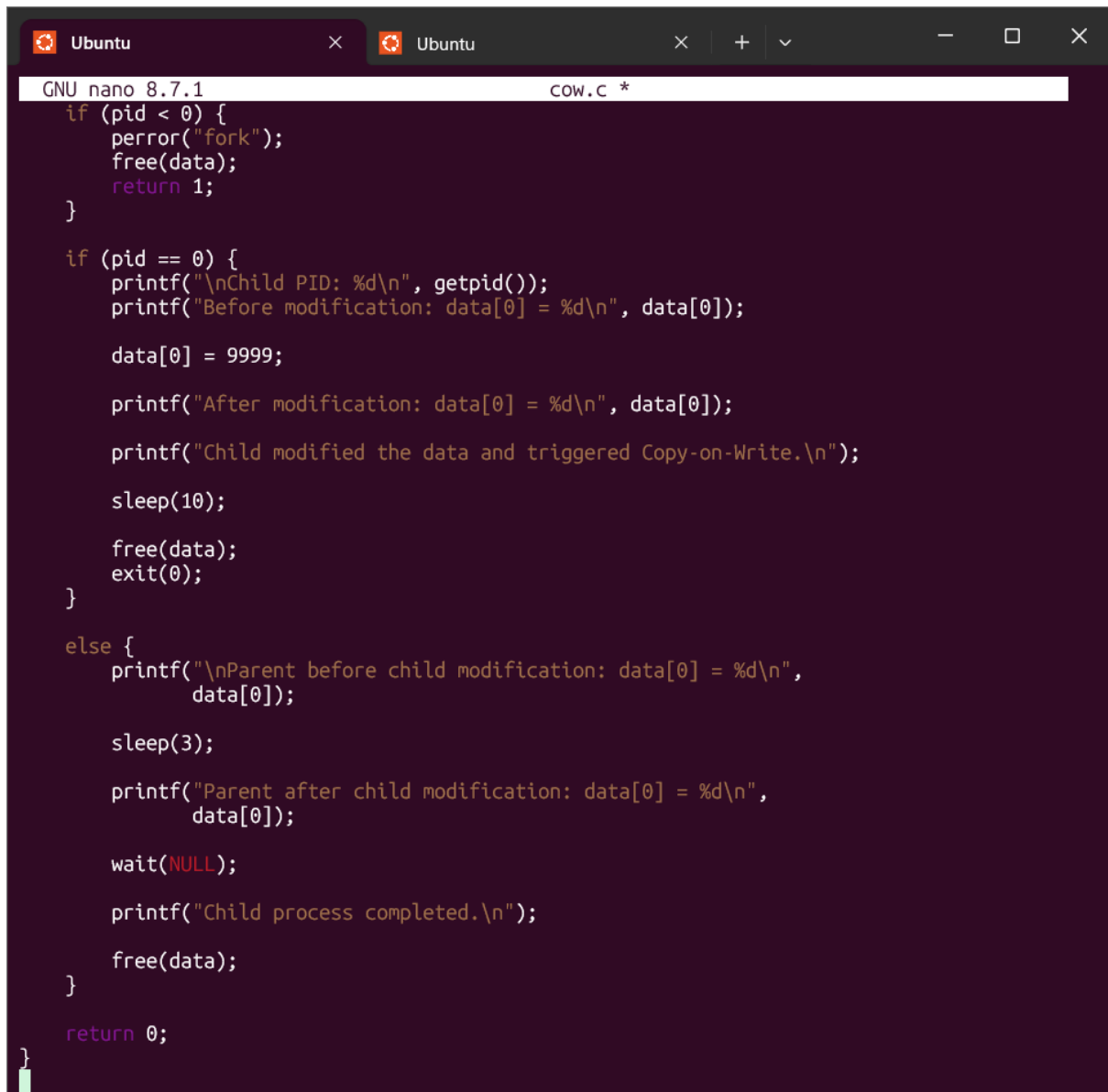
    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild PID: %d\n", getpid());
        printf("Before modification: data[0] = %d\n", data[0]);

        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);

        printf("Child modified the data and triggered Copy-on-Write.\n");
    }
}
```



The image shows a terminal window with two tabs, both labeled 'Ubuntu'. The active tab displays the GNU nano 8.7.1 editor editing a file named 'COW.C *'. The code is a C program that demonstrates the Copy-on-Write (COW) mechanism using `fork()`. It includes headers `<stdio.h>` and `<unistd.h>`. The program defines a global array `data` with the value 9999. The `main` function forks a child process. The child process prints its PID, the value of `data[0]` before modification, changes `data[0]` to 9999, prints the value after modification, and then sleeps for 10 seconds before freeing `data` and exiting. The parent process prints the value of `data[0]` before the fork, sleeps for 3 seconds, prints the value after the child's modification, waits for the child to complete, prints a completion message, and finally frees `data` and returns 0. The code is as follows:

```
GNU nano 8.7.1 COW.C *
#include <stdio.h>
#include <unistd.h>

int data = 9999;

int main() {
    if (pid < 0) {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0) {
        printf("\nChild PID: %d\n", getpid());
        printf("Before modification: data[0] = %d\n", data[0]);

        data[0] = 9999;

        printf("After modification: data[0] = %d\n", data[0]);

        printf("Child modified the data and triggered Copy-on-Write.\n");

        sleep(10);

        free(data);
        exit(0);
    }

    else {
        printf("\nParent before child modification: data[0] = %d\n",
            data[0]);

        sleep(3);

        printf("Parent after child modification: data[0] = %d\n",
            data[0]);

        wait(NULL);

        printf("Child process completed.\n");

        free(data);
    }

    return 0;
}
```

```
nocturnal@nocturnal:~$ nano cow.c
nocturnal@nocturnal:~$ nano cow.c
nocturnal@nocturnal:~$ gcc cow.c -o cow
nocturnal@nocturnal:~$ ./cow
Parent PID: 2104
Memory allocated: 4000000 bytes

Parent before child modification: data[0] = 0

Child PID: 2105
Before modification: data[0] = 0
After modification: data[0] = 9999
Child modified the data and triggered Copy-on-Write.
Parent after child modification: data[0] = 0
Child process completed.
nocturnal@nocturnal:~$
```